# Python Classes, Objects and Exception Handling

## Classes and __init__

A class defines a blueprint for objects. The __init__ method runs when an object is created and sets up instance attributes. The first parameter of every instance method is self, which refers to the object. Forgetting self causes TypeError: takes 0 positional arguments but 1 was given.

```
class Student:
    def __init__(self, name):
        self.name = name
        self.marks = []

    def add_mark(self, m):
        self.marks.append(m)

s = Student("Asha")
s.add_mark(90)
```

Attributes created in __init__ belong to each instance. Attributes defined directly in the class body are shared by all instances. A mutable class attribute such as a list is shared and causes the same kind of surprise as a mutable default argument.

## Inheritance

A subclass is written class Child(Parent). It inherits methods and attributes and may override them. Call super().__init__(...) inside the child __init__ so the parent setup still runs. Forgetting this leaves parent attributes undefined and later raises AttributeError.

```
class Person:
    def __init__(self, name):
        self.name = name

class Teacher(Person):
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject
```

## Special methods

__str__ returns a readable string for print(). __repr__ returns an unambiguous string for debugging. __len__, __eq__ and __lt__ let objects work with len(), == and sorting.

## try, except, else, finally

Wrap code that might fail in a try block and handle specific exceptions in except blocks. The else block runs if no exception happened. The finally block always runs and is used for cleanup such as closing files. Catching a bare except hides real bugs, so name the exception types you expect.

```
try:
    f = open("data.txt")
    data = f.read()
except FileNotFoundError:
    print("File not found")
except PermissionError:
    print("No permission")
else:
```

```
    print("Read", len(data), "characters")
finally:
    print("done")
```

## Raising exceptions

Use raise to signal an error condition, for example raise ValueError('age must be positive'). Custom exception classes inherit from Exception. Reading the traceback from the bottom up shows the exception type and message first, then the line where it happened.

## Working with files

Use the with statement so the file closes automatically, even if an error occurs. Open in mode 'r' to read, 'w' to overwrite, and 'a' to append. FileNotFoundError means the path is wrong relative to the current working directory.

```
with open("notes.txt", "w") as f:
    f.write("hello")

with open("notes.txt") as f:
    for line in f:
        print(line.strip())
```